

Project 주간보고서

학부(전공)/학과		응용소프트웨어공학과		작성일		2026. 05. 24 (12주차)	
팀명		이음		작성자 성명		홍효정 (서명)	
프로젝트명		캡스톤디자인II		담당(지도)교수		김삼문	
항목		내용					비고
이번 주 수행 내용		공통	● AI OCR결과 Split-View 출력 ● 문서 데이터 추출 로직				회의록 첨부
		홍효정 (팀장)	● 변경사항에 대한 UI/UX 세부조정 ● API부하 테스트 및 빌드 테스트				회의록 첨부
		박정열	● AI OCR결과 Split-View 출력 ● 문서 데이터 추출 로직 설계 ● 추출된 데이터 기반 자동 문서화 로직 설계				-
다음 주 수행 계획		공통	● 세부 조정 및 프로젝트 마무리				-
		홍효정 (팀장)	● ppt제작 - UI/UX상의 설계 초점 위주 내용 작성				-
		박정열	● ppt제작 - AI를 이용한 비전, 데이터 추출, 데이터의 전처리에 초점 두어 내용 작성				-
기타							
담당교수 지시사항							확인 날인
프로젝트 지도교수 지시사항		김삼문					확인 날인
							

※회의록 및 회의 증빙 사진 첨부

Project 팀 회의록

		과목명	캡스톤디자인II
회의 주제	프로젝트 구현 과정 소스		
학부(전공)/학과	응용소프트웨어공학과		
회의 일시	2026. 05. 24 12:05 ~ 12:35	회의 장소	온라인 회의
참석자	팀장 홍효정 (서명)	팀원 박정열 (서명)	
회의 내용			
주요 안건	1. AI OCR 사진 데이터 추출 Split-View 출력 2. AI OCR 문서 데이터 추출 Split-View 출력 3. 추출된 데이터 기반 자동 문서화 로직 설계		
● 이전 진행 내용 - AI를 이용한 비전 로직 구축 - 위험 코드 분류 및 그에 따른 기능 작성 - 객체 탐지 및 위험 분류 정상 작동 테스트 ● 안건 1 AI OCR 사진 데이터 추출 Split-View 출력 <pre># [기능 5] 문서 사진 AI 스캔 및 추출 (다중 품목 + 구분 추가) @app.post("/scan-document") async def scan_document(file: UploadFile = File(...)): try: encoded_image = await encode_image(file) response = await openai_client.chat.completions.create(model="gpt-4o-mini", messages=[{ "role": "system", # 💡 변경: 'type' 키를 추가하고 입고/출고 값을 추출하라고 지시 "content": "You are an expert at extracting information from logistics documents. Extract ALL items from the document. Output strictly as a JSON object containing an 'items' array. Each object in the array must have keys: 'company_name', 'item_name', 'quantity', and 'type'. For 'type', determine if it is receiving (입고) or issuing (출고).", }, { "role": "user", "content": [# 💡 변경: 유저 프롬프트에도 구분(입고/출고)을 명시 {"type": "text", "text": "이 이미지에서 업체명, 품목명, 수량, 구분(입고 또는 출고)을 모두 찾아서 배열 형태로 추출해줘. 수량은 숫자만."}, {"type": "image_url", "image_url": f"data:image/jpeg;base64,{encoded_image}"},], },], response_format={ "type": "json_object" }) result_json = json.loads(response.choices[0].message.content) items = result_json.get("items", []) formatted_items= [] for item in items: formatted_items.append({</pre>			

```

        "company_name": item.get("company_name", ""),
        "item_name": item.get("item_name", ""),
        "quantity": str(item.get("quantity", "0")),
        "type": item.get("type", "입고") # 💡 구분 값 추출 (없으면 기본값 '입고')
    })

    return {"items": formatted_items}
except Exception as e:
    print(f"OCR Error: {e}")
    raise HTTPException(status_code=500, detail=f"AI 인식 오류: {str(e)}")

```

● 안건 2 AI OCR 문서 데이터 추출 Split-View 출력

```

# [기능 6] 문서 파일 업로드 및 파싱 (다중 품목 + 구분 추가)
@app.post("/upload-excel")
async def upload_excel(file: UploadFileFile(...)):
    try:
        contents = await file.read()
        if file.filename.endswith('.csv'):
            df = pd.read_csv(io.BytesIO(contents))
        elif file.filename.endswith(('.xlsx', '.xls')):
            df = pd.read_excel(io.BytesIO(contents))
        else:
            raise HTTPException(status_code=400, detail="지원하지 않는 파일 형식입니다.")

        if df.empty:
            raise HTTPException(status_code=400, detail="파일이 비어있습니다.")
        # 1. 엑셀 데이터를 AI가 읽기 좋은 텍스트(JSON/Dict)로 변환
        raw_data = df.fillna("").to_dict(orient="records")
        # 2. AI에게 데이터 매핑 요청
        # 데이터가 너무 많을 경우를 대비해 상위 일정량만 보내거나 조절 가능합니다.
        response = await openai_client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[
                {
                    "role": "system",
                    "content": """
당신은 물류 데이터 전문가입니다.
제공된 원본 데이터(List of Dict)를 분석하여 우리 시스템 규격에 맞게 변환하세요.

변환 규칙:
1. 결과는 반드시 'items'라는 키를 가진 JSON 객체여야 합니다.
2. 각 항목은 다음 키를 가져야 합니다:
- 'company_name': 업체명, 상호, 공급처 등과 관련된 값
- 'item_name': 품목, 상품명, 모델명 등과 관련된 값
- 'quantity': 수량, 개수 등 (숫자만 남길 것)
- 'type': '입고' 또는 '출고' (데이터에 명시되지 않았다면 문맥상 판단하거나 기본값 '입고')
3. 원본 데이터의 컬럼명이 무엇이든 문맥을 파악해서 위 키값으로 매핑하세요.
            """
                },
                {
                    "role": "user",
                    "content": f"이 데이터를 규격에 맞춰 변환해줘: {json.dumps(raw_data, ensure_ascii=False)}"
                }
            ],
            response_format={"type": "json_object"}
        )
        # 3. AI 응답 파싱
    
```

```

result_json = json.loads(response.choices[0].message.content)
parsed_items = result_json.get("items", [])
# 4. 최종 데이터 보정 (안전장치)
formatted_items = []
for item in parsed_items:
    formatted_items.append({
        "company_name": str(item.get("company_name", "")),
        "item_name": str(item.get("item_name", "")),
        "quantity": str(item.get("quantity", "0")),
        "type": item.get("type", "입고")
    })

return {"items": formatted_items}
except Exception as e:
    print(f"AI Excel Parsing Error: {e}")
    raise HTTPException(status_code=500, detail=f"AI 문서 해석 실패: {str(e)}")

# [기능 7] 최종 검수 데이터 저장 (구분 값 동적 저장)
@app.post("/save-logistics")
async def save_logistics(
    company_id: strForm(...),
    vendorName: strForm(...),
    itemName: strForm(...),
    type: strForm(...),
    quantity: intForm(...),
    file: UploadFileFile(None)
):
    try:
        image_url = None
        if file:
            file_extension = file.filename.split(".")[-1] if "." in file.filename else "png"
            unique_filename = f"{uuid.uuid4()}.{file_extension}"
            file_bytes = await file.read()
            supabase.storage.from_("documents").upload(
                path=unique_filename,
                file=file_bytes,
                file_options={"content-type": file.content_type}
            )

            image_url = supabase.storage.from_("documents").get_public_url(unique_filename)
        doc_res = supabase.table("logistics_documents").insert({
            "company_id": company_id,
            "vendor_name": vendorName,
            "document_date": "now()",
            "raw_image_url": image_url
        }).execute()

        if not doc_res.data:
            raise Exception("Document insert failed")

        new_doc_id = doc_res.data[0]["id"]
        log_res = supabase.table("inventory_logs").insert({
            "company_id": company_id,
            "document_id": new_doc_id,
            "item_name": itemName,
            "type": type,
            "quantity": quantity,
            "unit": "EA"
        }).execute()
        return {"status": "success", "image_url": image_url}

```

```

except Exception as e:
    print(f"Error detail: {str(e)}")
    raise HTTPException(status_code=500, detail=f"DB 저장 오류: {str(e)}")

# [기능 8] 회사 데이터 가져오기 (수정본)
@app.get("/logistics-list")
def get_logistics_list(company_id: str):
    try:
        # 1. inventory_logs 테이블에서 필요한 컬럼들을 선택
        # 2. .select("...", logistics_documents(vendor_name)) 문법으로 조인 수행
        # 3. 정렬 기준을 실제 DB 컬럼인 'logged_at'으로 변경
        response = supabase.table("inventory_logs") \
            .select("""
                id,
                logged_at,
                item_name,
                type,
                quantity,
                logistics_documents (
                    vendor_name
                )
            """) \
            .eq("company_id", company_id) \
            .order("logged_at", desc=True) \
            .execute()

        return {"items": response.data}
    except Exception as e:
        # 구체적인 에러 확인을 위해 서버 터미널에 에러 로그 출력
        print(f"블러오기 에러 상세: {str(e)}")
        raise HTTPException(status_code=500, detail=f"데이터 블러오기 실패: {str(e)}")

```

● 안건 3 추출된 데이터 기반 자동 문서화 로직 설계

```

# [기능 9] 데이터 기간으로 필터링해 엑셀파일 변환
@app.get("/download-logistics")
async def download_logistics(
    start_date: str,
    end_date: str,
    company_id: str, # 💡 프론트엔드에서 넘겨받아야 함
    format: str="excel"
):
    try:
        # 1. DB에서 날짜 범위 및 회사 ID에 해당하는 데이터 실제 조회
        # .gte(크거나 같음), .lte(작거나 같음) 필터 사용
        response = supabase.table("inventory_logs") \
            .select("""
                logged_at,
                item_name,
                type,
                quantity,
                logistics_documents (
                    vendor_name
                )
            """) \
            .eq("company_id", company_id) \
            .gte("logged_at", f"{start_date}00:00:00") \
            .lte("logged_at", f"{end_date}23:59:59") \

```

```

        .order("logged_at", desc=False) ✕
        .execute()
    # 2. 데이터 유무 확인
    if not response.data:
        # 데이터가 없을 경우 404 혹은 빈 엑셀을 내려줄 수 있음
        raise HTTPException(status_code=404, detail="선택한 기간에 해당하는 데이터가 없습
니다.")
    # 3. 엑셀/CSV용 데이터 가공 (평면화)
    processed_data= []
    for row in response.data:
        processed_data.append({
            "날짜": row['logged_at'][:10] if row['logged_at'] else "-",
            "품목명": row['item_name'],
            "구분": "출고" if row['type'] == "OUT" else "입고",
            "수량": row['quantity'],
            "업체명": row.get('logistics_documents', {}).get('vendor_name', '-') if
row.get('logistics_documents') else "-"
        })

    # 4. Pandas DataFrame 변환
    df = pd.DataFrame(processed_data)

    if format == "excel":
        output = io.BytesIO()
        with pd.ExcelWriter(output, engine='openpyxl') as writer:
            df.to_excel(writer, index=False, sheet_name='Logistics_Report')
        output.seek(0)

        headers= {'Content-Disposition': f'attachment;
filename="logistics_{start_date}_{end_date}.xlsx"}
        return StreamingResponse(
            output,
            headers=headers,
            media_type='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
        )

    else: # CSV 처리
        stream = io.StringIO()
        df.to_csv(stream, index=False, encoding='utf-8-sig') # 엑셀 한글 깨짐 방지 위해
utf-8-sig 사용
        return StreamingResponse(
            iter([stream.getvalue()]),
            media_type="text/csv",
            headers={"Content-Disposition": f'attachment;
filename=logistics_{start_date}_{end_date}.csv"}
        )

    except Exception as e:
        print(f"Download Error: {str(e)}") # 서버 로그 확인용
        raise HTTPException(status_code=500, detail=f"파일 생성 실패: {str(e)}")

```

- 차기 회의 주제 안내
- 세부 조정 및 프로젝트 마무리

[별첨] 회의 증빙 사진

